

Job Scheduling

Code:

```
//CH.SC.U4CSE24135
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    int id;
    int deadline;
    int profit;
} Job;

void jobScheduling(Job jobs[], int n)
{
    // Sort jobs by profit (descending order)
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (jobs[j].profit < jobs[j + 1].profit)
            {
                Job temp = jobs[j];
                jobs[j] = jobs[j + 1];
                jobs[j + 1] = temp;
            }
        }
    }

    int maxDeadline = 0;
    for (int i = 0; i < n; i++)
    {
        if (jobs[i].deadline > maxDeadline)
        {
            maxDeadline = jobs[i].deadline;
        }
    }
}
```

```

int *result = (int *)malloc(maxDeadline * sizeof(int));
int *slot = (int *)malloc(maxDeadline * sizeof(int));
int totalProfit = 0;

for (int i = 0; i < maxDeadline; i++)
{
    slot[i] = 0;
}

for (int i = 0; i < n; i++)
{
    for (int j = jobs[i].deadline - 1; j >= 0; j--)
    {
        if (slot[j] == 0)
        {
            result[j] = i;
            slot[j] = 1;
            totalProfit += jobs[i].profit;
            break;
        }
    }
}

printf("\nJob Sequence: ");
for (int i = 0; i < maxDeadline; i++)
{
    if (slot[i])
    {
        printf("%d ", jobs[result[i]].id);
    }
}

printf("\nTotal Profit: %d\n", totalProfit);

free(result);

```

```

    free(slot);
}

int main()
{
    printf("CH.SC.U4CSE24135\n");

    int n;
    printf("Enter number of jobs: ");
    scanf("%d", &n);

    Job jobs[n];

    for (int i = 0; i < n; i++)
    {
        jobs[i].id = i + 1;
        printf("Enter profit and deadline for Job %d: ", jobs[i].id);
        scanf("%d %d", &jobs[i].profit, &jobs[i].deadline);
    }

    jobScheduling(jobs, n);

    return 0;
}

```

Output:

```

[keermanipamisetty@Keermanis-MacBook-Air HASSELL LAB % gcc job_scheduling.c -o job_scheduling
keermanipamisetty@Keermanis-MacBook-Air HASSELL LAB % ./job_scheduling
CH.SC.U4CSE24135
Enter number of jobs: 5
Enter profit and deadline for Job 1: 4 2
Enter profit and deadline for Job 2: 6 1
Enter profit and deadline for Job 3: 7 3
Enter profit and deadline for Job 4: 9 10
Enter profit and deadline for Job 5: 1 2

Job Sequence: 2 1 3 4
Total Profit: 26

```

Time Complexity:

The time complexity of this program is dominated by two main parts: sorting and job scheduling. First, the jobs are sorted in decreasing order of profit using Bubble Sort, which takes $O(n^2)$ time in the worst case. Next, the program finds the maximum deadline in $O(n)$ time. After that, for each job, it checks available time slots up to its deadline, which in the worst case can again take $O(n)$ time per job. Since this inner loop runs for all n jobs, it contributes $O(n^2)$ time in the worst case. Therefore, considering all steps and ignoring lower-order terms, the overall time complexity of the program is **$O(n^2)$** .

Space Complexity:

The space complexity of the program comes from the additional arrays used for scheduling. Two dynamic arrays, result and slot, are allocated with size equal to the maximum deadline. In the worst case, the maximum deadline can be equal to n . Therefore, the extra space used is proportional to n . Apart from these arrays, only a few integer variables are used, which require constant space. Hence, the overall space complexity of the program is **$O(n)$** .